might use databases to store data or communicate with other backend apps in different fashions.

You can imagine, in a real-world scenario, that the app would be the backend application managing the transactions in your bank accounts. Users may access their accounts and manage them via a web application (online banking) or a mobile app. Both the mobile apps and the web apps represent clients for the backend application. To manage users' transactions, the backend application needs to communicate with other backend solutions, and part of the data it manages needs to be persisted in a database. In figure 1.5, you can visualize the architecture of such a system.

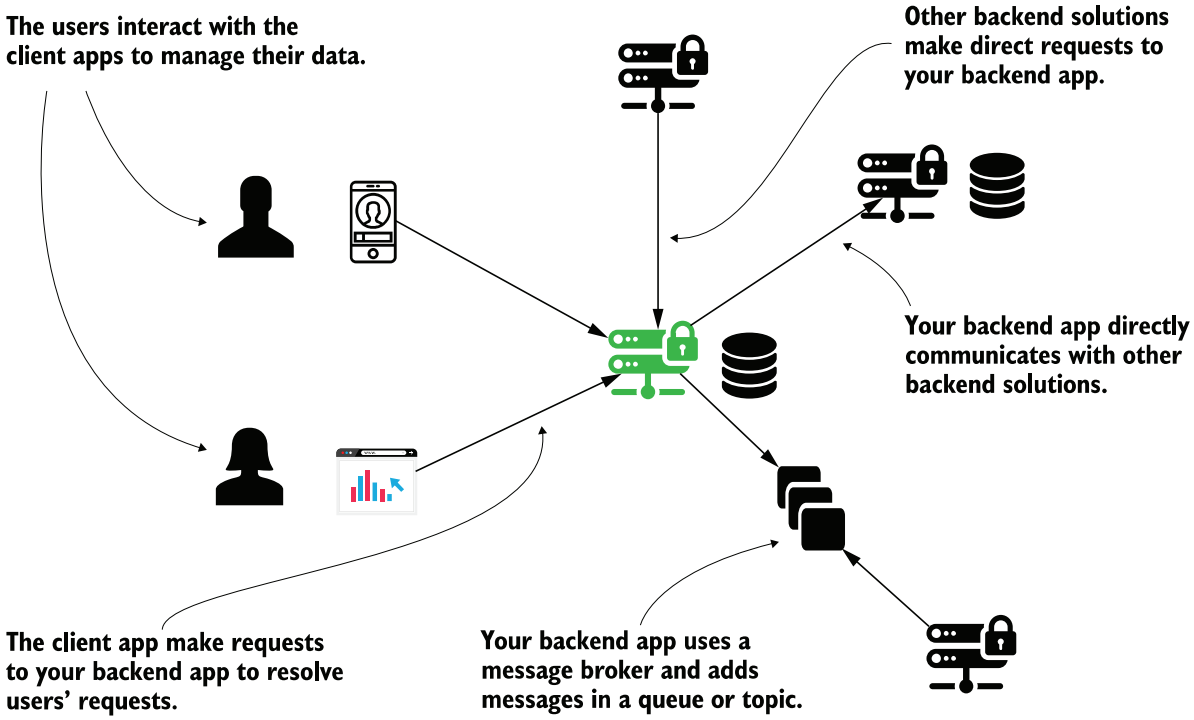

Figure 1.5 A backend app interacts in several ways with other apps and uses databases to manage data. Usually, a backend app is complex and may require the use of various technologies. Frameworks simplify the implementation by providing tools you can use to implement the backend solution faster.

> **NOTE** Don't worry if you don't understand all the details of figure 1.5. I don't expect you to know what a message broker is and not even how to establish the data exchange among the components. What I want you to see is that such a system can become complex in the real world and then understand that projects from the Spring ecosystem were built to help you eliminate this complexity as much as possible.

Spring offers an excellent set of tools for implementing backend applications. It makes your life easier with the different functionalities you generally implement in a backend solution, from integration with other apps to persistence in various database technologies. It's no wonder developers often use Spring for such applications. The

framework basically offers you everything you need in such implementations and is an excellent fit for any kind of architectural style. Figure 1.6 indicates the possibilities of using Spring for a backend app.

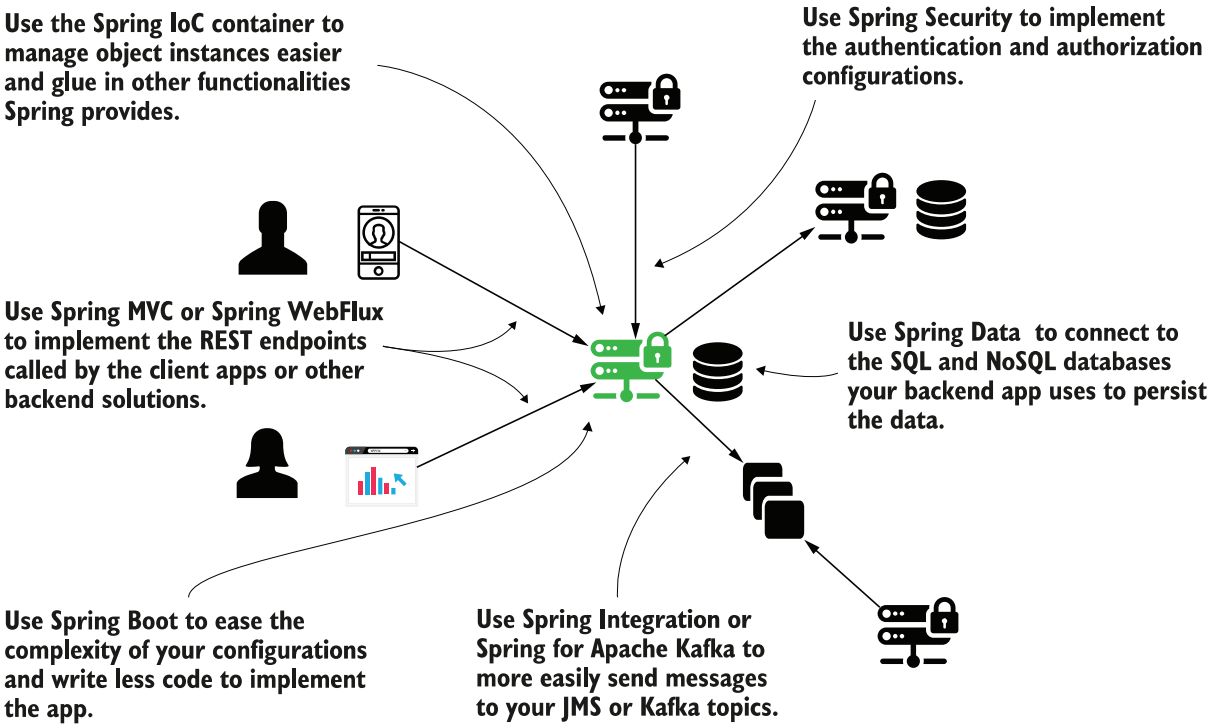

Figure 1.6 The possibilities of using Spring in a backend application are endless, from exposing functionalities that other applications can call to managing the database access, and from securing the application to managing integration though third-party message brokers.

1.3.2 Using Spring in a automation test app

Nowadays, we often use automation testing for end-to-end testing of systems we implement. Automation testing refers to implementing software that development teams use to make sure an application behaves as expected. A development team can schedule the automation testing implementation to frequently test the app and notify the developers if something is wrong. Having such functionality gives developers confidence because they know they'll be notified if they break anything in the existing capabilities of the app while developing new features.

While with small systems you can do the testing manually, it's always a good idea to automate the test cases. For more complex systems, manually testing all the flows isn't even an option. Because the flows are so numerous, it'd require a massive number of hours and too much energy to cover it completely.

It turns out that the most efficient solution is to have a separate team implement an app that has the responsibility of validating all the flows of the tested system. While developers add new functionalities to the system, this testing app is also enhanced to cover what's new, and the teams use it to validate that everything still works as desired. The developers eventually use an integration tool and schedule the app to run regularly to get feedback as soon as possible for their changes (figure 1.7).

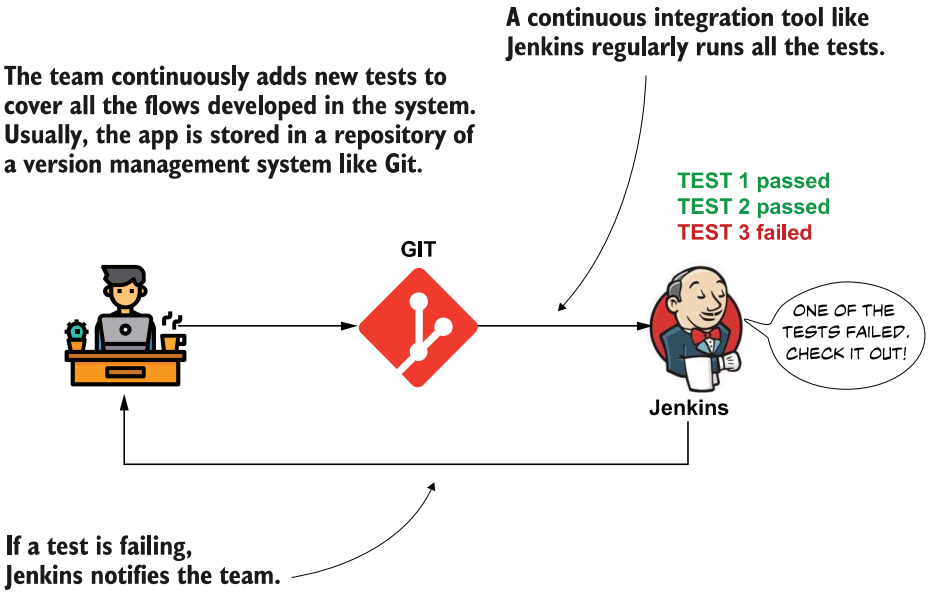

Figure 1.7 The team deploys the testing app in a test environment. A continuous integration tool like Jenkins executes the app regularly and sends feedback to the team. This way, the team is always aware of the system's status, and they know if they break something during development.

Such an application might become as complex as a backend app. In order to validate the flows, the app needs to communicate with the components of the system and even connect to databases. Sometimes the app mocks external dependencies to simulate different execution scenarios. For writing the test scenarios, developers use frameworks like Selenium, Cucumber, Gauge, and others. But, together with these frameworks, the app could still benefit in several ways from Spring's tools. For example, the app could manage the object instances to make the code more maintainable using the Spring IoC container. It could use Spring Data to connect to the databases where it needs to validate the data. It could send messages to queues or topics of a broker system to simulate specific scenarios or simply use Spring to call some REST endpoints (figure 1.8). (Remember, it's okay if this looks too advanced; meaning will be clarified as you progress through the book).

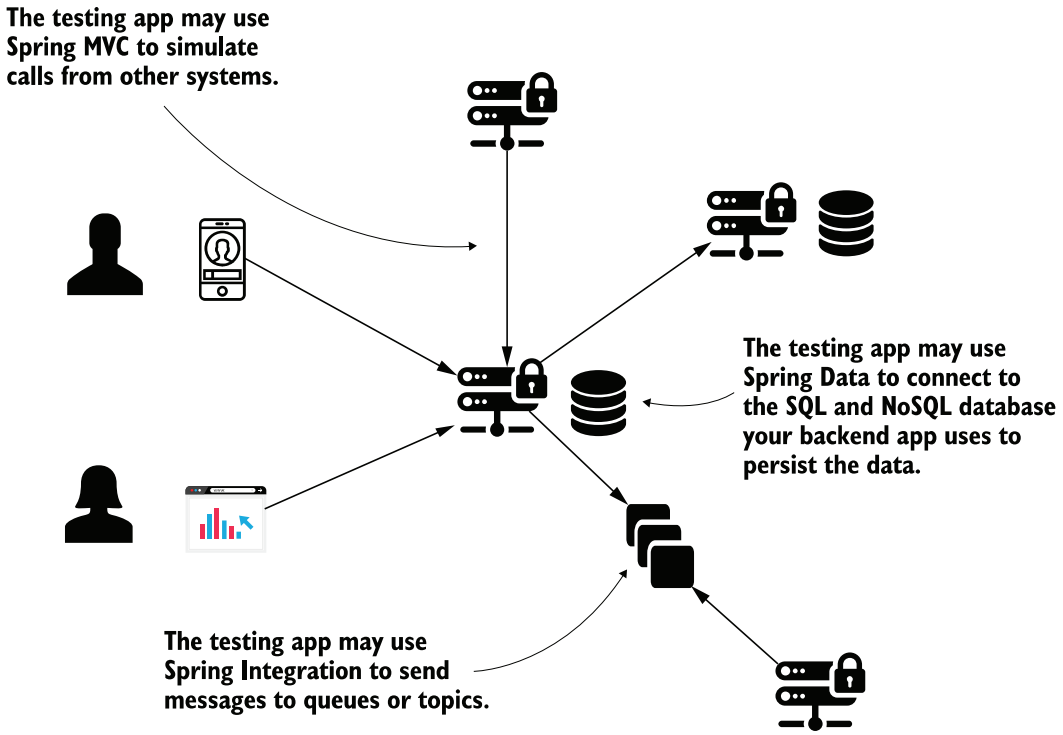

Figure 1.8 A testing app might need to connect to databases or communicate with other systems or the tested system. The developers can use components of the Spring ecosystem to simplify the implementations of these functionalities.

1.3.3 Using Spring for the development of a desktop app

Today, desktop applications are not that frequently developed, as web or mobile apps have taken the role of interacting with the user. However, there's still a small number of desktop applications, and components of the Spring ecosystem could be a good choice in the development of their features. A desktop app could successfully use the Spring IoC container to manage the object instances. This way, the app's implementation is cleaner and improves its maintainability. Additionally, the app could potentially use Spring's tools to implement different features, for example to communicate with a backend or other components (calling web services or using other techniques for remote calls) or implement a caching solution.

1.3.4 Using Spring in mobile apps

With its Spring for Android project (https://spring.io/projects/spring-android), the Spring community tries to help the development of mobile applications. Even though you'll probably rarely encounter this situation, it's worth mentioning that you can use Spring's tools to develop Android apps. This Spring project provides a REST client for Android and authentication support for accessing secured APIs.

1.4 When not to use frameworks

In this section, we discuss why you should sometimes avoid using frameworks. It's essential you know when to use a framework and when to avoid using them. Sometimes, using a tool that's too much for the job might consume more energy and also obtain a worse result. Imagine using a chainsaw to cut bread. While you could try to and even achieve a final result, it'd be more difficult and energy-consuming than using a regular knife (and you may end up with nothing but breadcrumbs instead of sliced bread). We'll discuss a few scenarios in which using a framework isn't a great idea, and then I'll tell you a story about a team I was part of that failed in the implementation of an app because of using a framework.

It turns out that, like everything else in software development, you shouldn't apply a framework in all cases. You'll find situations in which a framework is not a good fit—or maybe a framework is a good fit, but not the Spring framework. In which of the following scenarios should you consider not using a framework?

1. You need to implement a particular functionality with a footprint as small as possible. By footprint, I mean the storage memory occupied by the app's files.
2. Specific security requirements force you to implement only custom code in your app without making use of any open source framework.
3. You'd have to make so many customizations over the framework that you'd write more code than if you'd simply not used it at all.
4. You already have a functional app, and by changing it to use a framework you don't gain any benefit.

Let's discuss these points in more detail.

1.4.1 You need to have a small footprint

For point one, I refer to situations in which you need to make your application small. In today's systems, we find more and more cases in which the services are delivered in containers. You've likely heard about containers, such as Docker, Kubernetes, or other terms related to this subject (if not, again, that's okay).

Containers in their entirety is a topic beyond the scope of this book, so for now the only thing I need you to know is that when you use such a deployment fashion, you want your application to be as small as possible. A container is like a box in which your application lives. One crucial principle regarding app deployment in containers is that the containers should be easily disposable: they can be destroyed and recreated as fast as possible. The size of the app (footprint) matters a lot here. You can save seconds from the app initialization by making it smaller. That doesn't mean you won't use frameworks for all the apps deployed in containers.

But for some apps, which are usually also quite small, it makes more sense to improve their initialization and make their footprint smaller rather than adding dependencies to different frameworks. Such a case is a kind of application called *server-less function*. These server-less functions are tiny applications deployed in containers.

Because you don't have too much access to the way they're deployed, it looks like they execute without a server (hence their name). These apps need to be small, and that's why, for this specific case of apps, you'll want to avoid adding a framework as much as possible. Because of its size, it's also possible that you won't need a framework anyway.

1.4.2 Security needs dictate custom code

I said in point two that in specific situations, apps could not use frameworks because of security requirements. This scenario usually happens with apps in the field of defense or governmental organizations. Again, it doesn't mean all the apps used in governmental organizations are prohibited from using frameworks, but for some, restrictions are applied. You may wonder why. Well, say an open source framework like Spring is used. If someone finds a specific vulnerability, it will become known, and a hacker could use this knowledge to exploit it. Sometimes, stakeholders of such apps want to make sure the chances of someone hacking into their system is as close to zero as possible. This could lead to even rebuilding a functionality instead of using it from a third-party source.

> **NOTE** Wait! Earlier I said that it's more secure to use an open source framework because if a vulnerability exists, someone will likely discover it. Well, if you invest enough time and money, you probably can achieve this yourself as well. In general, it's cheaper to use a framework, of course. And if you don't want to be extra cautious, it makes more sense to use a framework. But in some projects, the stakeholders really want to make sure no information becomes public.

1.4.3 Abundant existing customizations make a framework impractical

Another case (point three) in which you might want to avoid using a framework is when you'd have to customize its components so much that you end up writing more code than if it hadn't been used. As I specified in section 1.1, a framework provides you parts that you assemble with your business code to obtain an app. These components, provided by the framework, don't fit perfectly, and you need to customize them in different ways. It's perfectly normal to customize the framework's components and the style in which they assemble than if you'd developed the functionality from scratch. If you find yourself in such a situation, you have probably chosen the wrong framework (search for alternatives) or you shouldn't use a framework at all.

1.4.4 You won't benefit from switching to a framework

In point four, I mentioned that a potential mistake could be trying to use a framework to replace something that already exists and is working in an app. Sometimes we are tempted to replace an existing architecture with something new. A new framework appears, and it's popular, and everyone uses it, so why shouldn't we change our app as well to use this framework? You can, but you need to attentively analyze what you want to achieve by changing something that works. In some cases, like my story from section

1.1, it could be helpful to change your app and make it rely on a specific framework. As long as this change brings a benefit, do it! A reason could be that you want to make the app more maintainable, more performant, or more secure. But if this change doesn't bring you a benefit, and sometimes it might even bring incertitude, then, in the end, you might discover you invested the time and money for a worse result. Let me tell you a story from my own experience.

An avoidable mistake

Using frameworks isn't always the best choice, and I had to learn that the hard way. Years earlier, we were working on the backend of a web application. Times influence many things, including software architectures. The app was using JDBC to directly connect to an Oracle database. The code was quite ugly. Everywhere the app needed to execute a query on the database it opened a statement and then sent a query that was sometimes written on multiple rows. You might be young enough not to have encountered JDBC direct usage in apps, but trust me, it's a long and ugly code.

At that time, some frameworks using another methodology to work with the database were becoming more and more popular. I remember when I first encountered Hibernate. This is an ORM framework, which allows you to treat the tables and their relationships in a database as objects and relationships among objects. When used correctly, it enables you to write less code and more intuitive functionality. When misused, it may slow down your app, make the code less intuitive, and even introduce bugs.

The application we were developing needed a change. We knew we could improve that ugly JDBC code. In my mind, we could at least minimize the number of lines. This change would have brought great benefits to maintainability. Together with other developers, we suggested using a tool provided by Spring called `JdbcTemplate` (you'll learn this tool in chapter 12). But others strongly pushed the decision to use Hibernate. It was quite popular, so why not to use it? (Actually it still is one of the most popular frameworks of its kind, and you'll learn about integrating it with Spring in chapter 13.) I could see changing that code to a completely new methodology would be a challenge. Moreover, I could see no benefits. The change also implied a greater risk of introducing bugs.

Fortunately, the change started with a proof of concept. After a couple of months, lots of effort, and stress, the team decided to quit.

After analyzing our options, we finished the implementation using `JdbcTemplate`. We managed to write cleaner code by eliminating a large number of lines of code, and we didn't need to introduce any new framework for this change.

1.5 What will you learn in this book

Since you opened this book, I assume you're probably a software developer in the Java ecosystem who found out it's useful to learn Spring. The purpose of this book is to teach you the foundations of Spring, assuming you know nothing at all about frameworks and, of course, about Spring. When I say Spring, I refer to the Spring ecosystem, not just the core part of the framework.

When you finish the book, you will have learned how to do the following:

- Use the Spring context and implement aspects around objects managed by the framework.
- Implement the mechanism of a Spring app to connect to a database and work with the persisted data.
- Establish data exchange between apps using REST APIs implemented with Spring.
- Build basic apps that use the convention-over-configuration approach.
- Use best practices in the standard class design of a Spring application.
- Properly test your Spring implementations.

Summary

- An application framework is a set of common software functionalities that provides a foundational structure for developing an application. A framework acts as the skeletal support to build an application.
- A framework helps you build an app more efficiently by providing functionality that you assemble to your implementation instead of developing it yourself. Using a framework saves you time and helps ensure there are fewer chances of implementing buggy features.
- Using a widely known framework like Spring opens a door to a large community, which makes it more likely that others will faces similar problems. You then have an excellent opportunity to learn about how others solved something similar to an issue you need to address, which will spare you the time of individual research.
- When implementing an application, always think of all possibilities, including not using a framework. If you decide to use one or more frameworks, take into consideration all their alternatives. You should think about the purpose of the framework, who else is using it (how big the community is), and for how long it's been on the market (maturity).
- Spring is not just a framework. We often refer to Spring as "Spring framework" to indicate the core functionalities, but Spring offers an entire ecosystem formed of many projects used in application development. Each project is dedicated to a specific domain, and when implementing an app, you might use more of these projects to implement the functionality you desire. The projects of the Spring ecosystem we'll use in this book are as follows:
  - Spring Core, which builds the foundation of Spring and provides features like the context, aspects, and basic data access.
  - Spring Data, which provides a high-level, comfortable-to-use set of tools to implement the persistence layer of your apps. You'll find how easy it is to use Spring Data to work with both SQL and NoSQL databases.
  - Spring Boot, which is a project of the Spring ecosystem that helps you apply a "convention-over-configuration" approach.

- Quite often, learning materials (like books, articles, or video tutorials) offer examples with Spring only for backend applications. While it's true that it's widespread to use Spring with backend apps, you can use Spring with other kinds of apps as well, even in desktop applications and automation testing apps.